

Week04 | Lakehouse 底座——Iceberg 快照/演进/性能基线

把 Week03 的 ingest baseline 升级成有状态记忆的数据底座

Week03 让数据能够稳定进入系统；Week04 要解决的是：

这份数据未来能不能被回看、被解释、被绑定到某一次 release、某一轮评测和某一版索引。

本周主线可以压缩成一句话：

从 ingest correctness, 升级到 table state reproducibility。

本周实践默认继续对齐 OmniSupport Copilot 的当前工程主线：`infra/docker-compose.yml`、`pipelines/lakehouse/iceberg_schemas.py`、`pipelines/lakehouse/assets.py`、`pyproject.toml`、`runbooks/`。如果项目代码还没有落地某条真实命令，本周讲义只保留同步位置，不编造不可运行路径。

本周一句话

把 Week03 的“可采、可重跑、可补数”推进成可快照、可回看、可演进、可建立性能基线 的 Lakehouse 底座。

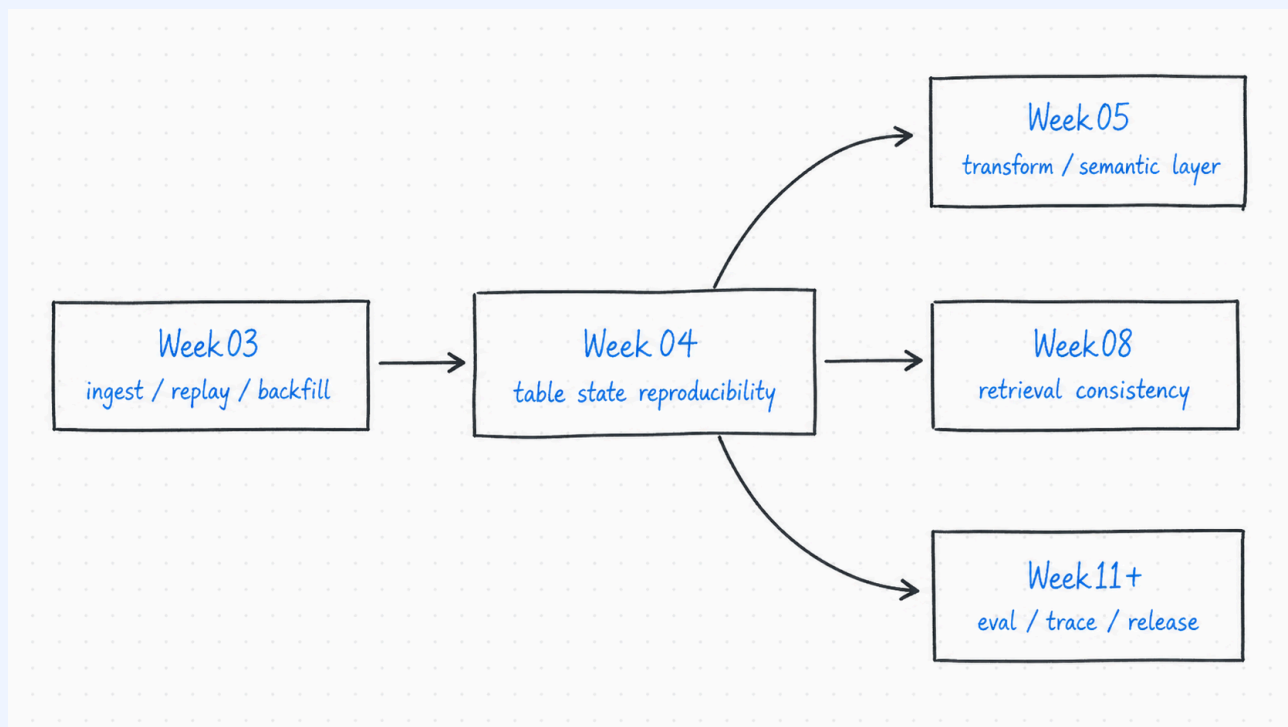
本周为什么重要

AI 系统上线后，最难复盘的往往不是“代码有没有改”，而是“回答背后的数据状态有没有变”。

- 同一个用户问题，今天答案变了，你需要知道是文档更新、工单状态变化、索引重建、prompt 变化，还是模型版本变化导致。
- 评测集跑分下降，你需要把分数绑定到某个 table snapshot，而不是只能说“可能数据变了”。
- RAG 索引需要回滚时，你不能只回滚代码，也要能回到当时那版 Bronze / Silver 数据状态。
- Week05 的 transform、Week06 的 asset factory、Week08 的 retrieval consistency, 都需要 Week04 提供稳定数据状态锚点。

Iceberg 在 Week04 里不是“新组件”，而是 AI 数据工程第一次把 状态记忆 落到数据实体层。

Week03 -> Week04 -> Week05 主线



这张图只表达一个判断：

Week04 不是 Week03 的附属实验，而是后续所有“可复盘、可回滚、可对比”的数据状态层。

本周做什么 / 不做什么

本周做

- 用 PyIceberg 建立本地最小 Lakehouse 闭环
- 解释 snapshot / manifest / metadata log / time travel
- 站住最小 4 表：2 张 Bronze、2 张 Silver
- 演示 add-column 级别 schema evolution
- 产出 baseline report 和 Week04 runbook

本周不做

- 不引入 Spark / Hive Metastore / Nessie / Trino 作为学生主线
- 不提前进入 Week05 的 dbt、语义层和指标口径
- 不提前进入 Week07-08 的 evidence anchor / RAG serving
- 不把 compaction、expire snapshots 做成必做 pipeline
- 不重写 OmniSupport Copilot 的项目代码

本周在整门课里的位置

上承 Week03

接住 manifest、state、checkpoint、replay / backfill 基线，把“入湖”推进成真正有记忆的表状态。

本周解决

解决 snapshot、history、metadata log、schema evolution、hidden partitioning 和 baseline report。

下接 Week05

Week05 会站在稳定的 Bronze / Silver 表上继续写 transform 和语义层，而不是重新解决数据状态问题。

影响后续

Week06 的 Dagster backfill、Week08 的 RAG consistency、Week11 之后的评测绑定，都会持续消费 Week04 的状态记忆层。

五个课时

Week04 | 课时 1 | 为什么 AI 数据工程需要“有记忆的表”，而不只是能查的表

解决的问题：为什么 raw bucket、Postgres 当前表、向量索引都不能单独承担坏案例复盘和 release 绑定。

你会产出：lakehouse_foundation_v1.md 与一组状态记忆复盘问题。

进入课时 1

Week04 | 课时 2 | Iceberg 的状态模型：snapshot、manifest、metadata log 与 time travel

解决的问题：Iceberg 靠什么把一组文件组织成可提交、可回看、可演进的表状态。

你会产出：snapshot_state_model_v1.md 与 metadata inspection 解读草稿。

进入课时 2

Week04 | 课时 3 | 从 Week3 入湖到 Week4 湖仓：Bronze / Silver 最小表设计与 hidden partitioning

解决的问题：为什么本周只做 bronze.raw_ticket_event、bronze.raw_doc_asset、silver.ticket_fact、silver.knowledge_doc。

你会产出：source-to-Iceberg mapping 与 4 表设计说明。

进入课时 3

Week04 | 课时 4 | Pylceberg 本地最小闭环：Catalog、Warehouse、写入、历史查看、Schema Evolution

解决的问题：为什么当前选择 Pylceberg + PostgreSQL-backed SQL Catalog + MinIO warehouse，而不是先拉起重型湖仓栈。

你会产出：catalog runtime plan 与 Week04 runbook 主干。

进入课时 4

Week04 | 课时 5 | 性能基线不是调优冲动：files / history / snapshots 视角下的 Week4 验收

解决的问题：什么是 baseline report，为什么它先于调优、也先于后续 transform / retrieval。

你会产出：iceberg_baseline_report.md。

进入课时 5

实验与作业的关系

Lab：跑通闭环

实验页关注“怎么证明最小闭环存在”：

- catalog 能加载
- warehouse 能写
- 最小 4 表能 materialize

- snapshots / history / files 能 inspect
- time travel 与 add-column evolution 有记录

Assignment: 整理交付包

作业页关注“怎么把结果交给团队继续用”:

- 目标与边界
- source-to-Iceberg mapping
- Bronze / Silver 表设计
- runtime plan
- runbook
- baseline report

[进入实验](#) · [进入作业](#)

本周交付物

工件	解决的问题
<code>docs/blueprints/week04/lakehouse_foundation_v1.md</code>	Week04 为什么存在、解决什么、不解决什么
<code>docs/blueprints/week04/source_to_iceberg_mapping_v1.md</code>	source 字段、类型、语义如何进入 Iceberg
<code>docs/blueprints/week04/bronze_silver_table_design_v1.md</code>	最小 4 表的角色、列设计、边界
<code>docs/blueprints/week04/catalog_runtime_plan_v1.md</code>	catalog / warehouse / table location / thin wrapper 方案
<code>runbooks/week04/README.md</code>	团队如何运行、排查、复核 Week04 闭环
<code>reports/week04/iceberg_baseline_report.md</code>	snapshots / files / history / schema evolution 的最终基线记录

本周最重要的工程判断

1. Week04 的核心不是“引入 Iceberg”，而是建立 table state reproducibility。
2. snapshot、time travel、schema evolution 不是高级功能，而是坏案例复盘和 release-aware 数据系统的基础能力。
3. Bronze / Silver 最小闭环没有站住之前，不应该提前跳进语义层、RAG 或治理。
4. 本周先求本地可跑、可解释、可验收，不求平台看起来重型。
5. baseline 不是 benchmark, 更不是调参; baseline 是团队以后判断变化是否可接受的第一份状态证据。